

Classes, Objects, and Interfaces

My teaching notes

A word before you read. These are my notes from over the years, cleaned up so they are readable. A few things to keep in mind:

- **They are not a study guide, and they are not gospel.** Not everything here appears in a lecture, and not every topic has notes yet; I will write more as we go.
- **I write from memory and revise every semester.** A typo or an imprecise statement can survive longer than it should.
- **I use AI to help** convert documents to markdown, check for issues, and add explanations. I do not trust it completely, so I am still editing these myself.
- **Java changes.** The basics we cover are pretty resilient, but some features do move over time.

So: if you find something that looks wrong, contradicts the book, or just does not behave the way I claim when you run it, tell me. You will be doing the other students a favor, and you will understand the material better for having caught it. When in doubt, the textbook is the tiebreaker, and the compiler beats us both.

These notes also go beyond the textbook in places, filling in gaps or covering things a little differently than the book does. They are not a replacement for the assigned chapters or for the recorded lectures, and neither of those replaces the notes either. The strongest preparation uses all three together: read the chapter, watch the lecture, and study the notes.

Examples mix ordinary, typical scenarios with a few pop culture ones, since seeing the same idea twice, once plain and once fun, tends to make it stick. Every listing compiles as written unless a comment says otherwise. Inheritance and polymorphism are not covered here beyond a short preview at the end; they get their own unit later in the semester.

1. Why Object-Oriented Programming

Every program you have written so far in this course keeps its data in variables, arrays, and `ArrayList` objects, and its logic in methods that operate on that data from the outside. That works, but it starts to strain as a program grows. Passing the same five or six related variables into every method, and hoping every method updates them consistently, becomes error-prone fast.

Object-oriented programming addresses this by bundling data and the behavior that operates on that data into a single unit, a class. This bundling is called **encapsulation**. A well-encapsulated class hides its internal data (its fields) behind a public set of methods, so the rest of the program cannot reach in and set the data to something invalid. If a car's mileage should never decrease, encapsulation is what makes that a rule the class itself enforces, rather than a rule every caller has to remember.

Three practical benefits follow from this:

- **Safety.** A class can refuse to allow an invalid state, since it controls every way its data can change.
- **Simplicity for the caller.** Code that uses a class only needs to know what its public methods do, not how they work internally.
- **Change without breakage.** The internal implementation of a class can change completely, and any code that only uses its public methods keeps working, as long as those methods still behave as promised.

2. The Three Characteristics of an Object

Every object, no matter what class it comes from, has three characteristics.

- **State.** An object's properties and current values, its internal condition at a given moment. In code, state lives in an object's fields (also called data members or attributes).
- **Behavior.** The actions or operations an object can perform, how it responds to interactions. In code, behavior lives in an object's methods. Behavior is sometimes called an object's *implementation*.
- **Identity.** What distinguishes one object from another, even if two objects have identical state. Identity is sometimes loosely called an object's "type," but type (its class) is only one part of identity; two `Car` objects can share the exact same class, make, model, and year, and still be two different cars. Java represents identity, in part, through a hash code, which is why overriding `hashCode` correctly matters once a class needs to be compared or looked up by value (Section 9).

```
Car civic1 = new Car("Honda", "Civic", 2022);
Car civic2 = new Car("Honda", "Civic", 2022);

System.out.println(civic1 == civic2);    // false -- two different objects
System.out.println(civic1 == civic1);    // true  -- the same object, compared to itself
```

Both cars share identical state (same make, model, year), yet `==` reports them as different. That is identity at work: `==` on objects compares whether two references point to the exact same object in memory, not whether their fields match. We come back to this distinction with `equals` in Section 9.

3. Anatomy of a Class

A class is a blueprint. It does not, by itself, hold any data; it describes what every object built from it will have. A typical class has four parts: fields, a constructor, methods, and often a mix of public getters and setters alongside private helper methods.

A first example

```
public class Car {

    // fields (data members) -- these hold each Car object's state
    private String make;
    private String model;
```

```

private int year;
private int mileage;

// constructor -- runs once, when an object is created with "new"
public Car(String make, String model, int year) {
    this.make = make;
    this.model = model;
    this.year = year;
    this.mileage = 0;          // every new car starts at zero miles
}

// getters -- public, read-only access to private fields
public String getMake()    { return make; }
public String getModel()   { return model; }
public int getYear()       { return year; }
public int getMileage()    { return mileage; }

// a setter, with its validation delegated to a private helper method
public void setMileage(int newMileage) {
    if (isValidMileage(newMileage)) {
        mileage = newMileage;
    } else {
        System.out.println("Ignored an invalid mileage: " + newMileage);
    }
}

// a private helper method -- only code inside this class can call it
private boolean isValidMileage(int candidate) {
    return candidate >= mileage;    // mileage should never run backward
}

// an ordinary behavior method, built on top of the setter above
public void drive(int miles) {
    setMileage(mileage + miles);
}
}

```

Walking through each part:

- **Fields.** `make`, `model`, `year`, and `mileage` are private. Nothing outside this class can read or write them directly; every interaction goes through a method.
- **Constructor.** A constructor has the same name as the class and no return type, not even `void`. It runs exactly once, when `new Car(...)` executes, and its job is to leave the new object in a valid starting state. The keyword `this` refers to the object currently being built, so `this.make = make;` means “store the parameter `make` into this object’s `make` field,” distinguishing the field from the parameter of the same name.
- **Getters and setters.** A getter returns a field’s value without allowing it to be changed. A setter allows a field to change, but only through a method the class controls, which is exactly where validation belongs.
- **Private helper methods.** `isValidMileage` is not part of the class’s public interface; it exists

purely to keep `setMileage` readable and to avoid repeating the same check elsewhere. A private helper can change its internal logic freely without affecting any code outside the class, since nothing outside the class is allowed to call it.

- **Ordinary methods.** `drive` is public behavior built out of the class's own private pieces (here, it reuses `setMileage` instead of touching `mileage` directly), which keeps the validation rule in exactly one place.

Try it yourself: sketch a `Wizard` class with fields for `name`, `house`, and `spellPower`, a constructor, getters, a `castSpell(String spellName)` method, and a private helper `hasEnoughPower()` that `castSpell` calls before allowing the spell.

4. Access Modifiers

An access modifier controls which code is allowed to see or call a field or method. CIS 3300 uses three of them regularly, with a fourth, `protected`, becoming important later once inheritance is introduced.

- **private.** Visible only inside the same class. This is the default choice for fields, so that all access goes through methods.
- **public.** Visible from anywhere. This is the usual choice for a class's constructor and the methods meant to form its public interface.
- **default (no modifier at all).** Visible to any class in the same package, but nowhere else. Writing nothing is a deliberate choice, not an oversight.
- **protected.** Visible in the same package, and also to subclasses in other packages. This modifier does very little for us until the inheritance unit, where subclasses become relevant.

Table 1: Access modifier visibility

Modifier	Same class	Same package	Subclass (other package)	World
<code>public</code>	Yes	Yes	Yes	Yes
<code>protected</code>	Yes	Yes	Yes	No
default	Yes	Yes	No	No
<code>private</code>	Yes	No	No	No

A practical rule of thumb for this unit: make fields `private`, make constructors and the methods other classes are meant to call `public`, and leave a method without a modifier, or mark it `private`, when it exists only to help the class do its own job.

5. Creating and Using Objects

A class is a blueprint; an object is one specific thing built from that blueprint. Creating an object is called **instantiation**, and it happens with the `new` keyword, which allocates memory for the object and then runs the matching constructor.

```
Car civic    = new Car("Honda", "Civic", 2022);
Car mustang = new Car("Ford", "Mustang", 2024);

civic.drive(120);
```

```

mustang.setMileage(15);

System.out.println(civic.getMake() + " " + civic.getModel()
    + ": " + civic.getMileage() + " miles");
System.out.println(mustang.getMake() + " " + mustang.getModel()
    + ": " + mustang.getMileage() + " miles");

```

Two objects built from the same class are completely independent. Calling `civic.drive(120)` changes only `civic`'s `mileage` field; `mustang` is untouched. This is the payoff of encapsulation from Section 1: each object manages its own state, and a program becomes a set of objects sending each other messages (calling each other's methods) rather than one long list of steps operating on shared variables.

6. Class Relationships: Uses-a and Has-a

Classes rarely work alone. Two relationships come up constantly in this unit, short of full inheritance, which is previewed in Section 11.

- **Dependence** (“uses-a”). One class uses another, typically by accepting it as a method parameter, without owning or storing it long term.
- **Aggregation** (“has-a”). One class contains an instance of another, usually as a field, and manages a collection of them over time.

```

import java.util.ArrayList;

// A Garage HAS-A list of Cars: aggregation
public class Garage {
    private String location;
    private ArrayList<Car> fleet;

    public Garage(String location) {
        this.location = location;
        this.fleet = new ArrayList<>();
    }

    public void addCar(Car car) {
        fleet.add(car);
    }

    public Car findByModel(String model) {
        for (Car car : fleet) {
            if (car.getModel().equalsIgnoreCase(model)) {
                return car;
            }
        }
        return null;    // not found
    }

    public void printFleet() {
        for (Car car : fleet) {
            System.out.println(car.getYear() + " " + car.getMake()

```

```

        + " " + car.getModel());
    }
}

// A Mechanic USES a Car, but does not own or store one: dependence
public class Mechanic {
    public void inspect(Car car) {
        System.out.println("Inspecting a " + car.getYear() + " " + car.getModel());
    }
}

```

Garage is our first multi-class program: a **Garage** object manages many **Car** objects, and code that wants to work with a specific car goes through the **Garage**, not around it. **Mechanic** shows the weaker relationship: a **Mechanic** object never stores a **Car** at all, it simply borrows one for the duration of a single method call.

A has-a relationship, specifically the kind shown here, is called aggregation. It is worth being precise about two of its properties, since they are easy to miss. First, it is **unidirectional**: a **Garage** holds a reference to its **Car** objects, but a **Car** does not hold a reference back to its **Garage**, or even know one exists. Second, the two objects have **independent lifecycles**: a **Car** can exist without ever being stored in a **Garage**, and removing a **Car** from a **Garage** does not destroy the **Car**, it only means the **Garage** no longer holds a reference to it. Java has no keyword dedicated to has-a; it falls directly out of using one class as the type of another class's field, which is also what makes it useful for code reuse: a **Garage** does not reimplement anything a **Car** already does, it simply holds **Car** objects and calls their methods. For a fuller treatment with a different example, see this [GeeksforGeeks](https://www.geeksforgeeks.org/java/what-is-has-a-relation-in-java/) article on the has-a relationship: <https://www.geeksforgeeks.org/java/what-is-has-a-relation-in-java/>

A quick note on terminology, since that article blurs this: aggregation and composition are not interchangeable, even though both express a has-a relationship. The difference is lifecycle dependency, not the has-a idea itself. In aggregation, the part can exist independently of the whole; a **Car** is fully valid on its own, before, after, or without ever belonging to a **Garage**, which is exactly what **Garage** and **Car** demonstrate above. In composition, the part's lifecycle is bound to the whole, typically because it is created inside the whole's own constructor and never handed out to other code; a **House** building its own **Room** objects is a common example. This unit only uses aggregation.

7. Passing Objects to Methods

Java passes every argument by value, including object references. That phrase is worth unpacking carefully, because it explains behavior that otherwise looks inconsistent.

```

public static void serviceCar(Car car) {
    car.setMileage(car.getMileage() + 5000);    // changes the object itself
}

public static void replaceCar(Car car) {
    car = new Car("Toyota", "Corolla", 2020);  // only reassigns the LOCAL copy
}

public static void main(String[] args) {
    Car civic = new Car("Honda", "Civic", 2022);
}

```

```

    serviceCar(civic);
    System.out.println(civic.getMileage());    // 5000 -- the change is visible

    replaceCar(civic);
    System.out.println(civic.getModel());      // "Civic" -- unchanged
}

```

What is copied when `civic` is passed into `serviceCar` is the reference (effectively, the object's address), not the object itself. Two references can point at the same object, so a change made *through* the parameter, such as calling a setter on it, is visible to the caller. However, reassigning the parameter itself, as `replaceCar` does, only points the local copy of the reference somewhere new; it has no effect on the caller's original variable, which still points to the original `Car`.

8. Objects in an ArrayList

Everything you already know about `ArrayList` from earlier in the semester still applies once the element type is a class you wrote instead of a primitive wrapper or a `String`. The only new habit is that printing or comparing elements now depends on methods your class does or does not override, which is exactly the subject of Section 9.

A typical example: continuing the Garage

```

Garage shop = new Garage("Main Street");
shop.addCar(new Car("Honda", "Civic", 2022));
shop.addCar(new Car("Ford", "Mustang", 2024));
shop.addCar(new Car("Toyota", "Corolla", 2020));

shop.printFleet();

Car found = shop.findByModel("Mustang");
if (found != null) {
    found.drive(50);
}

```

A pop culture example: a Pokedex

[*Pokemon*]

```

import java.util.ArrayList;

ArrayList<Pokemon> party = new ArrayList<>();
party.add(new Pokemon("Pikachu", "Electric", 35));
party.add(new Pokemon("Charizard", "Fire", 78));
party.add(new Pokemon("Snorlax", "Normal", 160));

for (Pokemon p : party) {
    System.out.println(p.getName() + " (" + p.getType()
        + "), HP: " + p.getHp());
}

```

```

int totalHp = 0;
for (Pokemon p : party) {
    totalHp += p.getHp();
}
System.out.println("Total party HP: " + totalHp);

```

Both examples do the same three things with a list of objects: build it up with `add`, walk it with an enhanced `for` loop, and pull a running total or a specific match out of it. The pattern does not change based on what the objects are.

9. Overriding `toString`, `equals`, and `hashCode`

Every class you write, whether you realize it or not, already has `toString`, `equals`, and `hashCode` methods, inherited automatically from Java’s built-in `Object` class. The defaults are rarely what you want.

- The default `toString` prints something like `Car@1b6d3586`, the class name and a hash code, which tells a human reading console output almost nothing useful.
- The default `equals` behaves exactly like `==`: it checks whether two references point to the same object, not whether two objects represent the same real-world thing. This is the identity behavior from Section 2.
- The default `hashCode` is typically derived from the object’s memory location, so two “equal-looking” objects usually get different hash codes unless you say otherwise.

Overriding all three, together, fixes this:

```

import java.util.Objects;

public class Car {
    private String make;
    private String model;
    private int year;
    private int mileage;

    // (constructor, getters, and setMileage from Section 3 omitted here)

    @Override
    public String toString() {
        return year + " " + make + " " + model + " (" + mileage + " miles)";
    }

    @Override
    public boolean equals(Object other) {
        if (this == other) {
            return true; // same object: trivially equal
        }
        if (!(other instanceof Car)) {
            return false; // not even a Car: cannot be equal
        }
        Car otherCar = (Car) other;

```

```

        return year == otherCar.year
            && make.equals(otherCar.make)
            && model.equals(otherCar.model);
    }

    @Override
    public int hashCode() {
        return Objects.hash(make, model, year);
    }
}

```

A rule that always holds in Java: **if you override equals, you must also override hashCode**, using the same fields. Classes such as `ArrayList`, and especially the hash-based collections you will see later, rely on this pairing to work correctly; breaking it produces bugs that are difficult to track down, since the code compiles and often runs, it just gives the wrong answer.

```

Car civic1 = new Car("Honda", "Civic", 2022);
Car civic2 = new Car("Honda", "Civic", 2022);

System.out.println(civic1 == civic2);           // false, as before -- different objects
System.out.println(civic1.equals(civic2));       // true, now that equals compares fields
System.out.println(civic1);                     // uses toString automatically

```

10. Interfaces

An interface declares a set of methods a class promises to implement, without providing a body for most of them. Where a class describes what an object *is*, an interface describes what an object *can do*.

```

public interface Serviceable {
    void service();                // abstract -- every implementer must write this

    default void honk() {          // default -- a ready-made body, used as-is
        System.out.println("Beep beep!");
    }
}

public class Car implements Serviceable {
    // ... fields, constructor, and other methods from earlier sections ...

    @Override
    public void service() {
        System.out.println(model + " has been serviced at " + getMileage() + " miles.");
    }
}

```

A class that writes `implements Serviceable` makes a promise: it must provide a body for `service()`, or the code will not compile. The `default` keyword is what makes `honk()` different: an interface method marked `default` already has a body, so an implementing class inherits it.

for free and only needs to override it if it wants different behavior. Default methods were added to Java specifically so that an interface could grow new methods over time without breaking every class that already implements it.

An interface can also declare constants. Any field written inside an interface is automatically **public**, **static**, and **final**, whether or not those keywords are written out, so it behaves as a single shared, unchangeable value that every implementing class can reference.

```
public interface Serviceable {
    int MAX_MILES_BETWEEN_SERVICES = 5000;    // implicitly public static final

    void service();

    default void honk() {
        System.out.println("Beep beep!");
    }
}
```

*A pop culture parallel [Superheroes]: a **Flyable** interface with an abstract **fly()** method and a default **announceFlight()** method that prints “Taking off!” Any class that implements **Flyable**, a superhero, a drone, a dragon, must supply its own **fly()**, but gets **announceFlight()** for nothing unless it chooses to override it too.*

A class can implement more than one interface, separated by commas, which is one reason interfaces are useful: a **Car** could implement both **Serviceable** and, say, a hypothetical **Insurable** interface, without either interface knowing about the other.

11. Where This Goes Next: A Preview of Inheritance

This unit built two kinds of relationships between classes: dependence (uses-a) and aggregation (has-a). There is a third kind, waiting in the wings: inheritance (is-a), where one class is a more specialized version of another and automatically receives its fields and methods.

- A **Garage** **has-a** list of **Car** objects (aggregation, Section 6).
- A **Mechanic** **uses-a** **Car** when it inspects one (dependence, Section 6).
- A **SportsCar** **is-a** **Car**, a more specialized kind of car that adds or overrides behavior, while still being usable anywhere a plain **Car** is expected (inheritance, not yet covered).

That third relationship is powerful enough to need a unit of its own, covering **extends**, method overriding for polymorphism, abstract classes, and how inheritance interacts with the access modifiers from Section 4, especially **protected**. For now, it is enough to recognize the vocabulary: uses-a and has-a are what this unit builds, and is-a is what comes next.